

An investigation of Technical Debt in Automatic Production Systems

Terese Besker

Chalmers University of Technology
Computer Science and Engineering
Gothenburg, Sweden
besker@chalmers.se

Antonio Martini

Chalmers University of Technology
Computer Science and Engineering
Gothenburg, Sweden
antonio.martini@chalmers.se

Jan Bosch

Chalmers University of Technology
Computer Science and Engineering
Gothenburg, Sweden
jan.bosch@chalmers.se

Matthias Tichy

Ulm University, Institute of Software
Engineering and Programming
Languages
Ulm, Germany
matthias.tichy@uni-ulm.de

ABSTRACT

Technical Debt is a recent concept, borrowed from the financial domain. It has been recently used in software development to describe technical sub-optimal solutions that have short-term benefits but long-term extra-costs. However, no body of literature investigates how Automatic Production Systems companies deal with Technical Debt. We investigated how Technical Debt is known, how much it hurts and how is managed in an automatic production systems company. Results from one in-depth investigation show that the automatic production systems company spend quite a lot of resources because of Technical Debt, both in the extra-costs (interest) and in its management. The company presents moderate awareness of what Technical Debt is and how much is present in its systems. However, the tracking level is quite low. We, therefore, claim that Technical Debt needs more research in this domain, as it is a source of substantial extra-costs and the current practices to manage it are not suitable.

Keywords

Technical Debt, Automatic Production Systems, management, survey

1. INTRODUCTION

Highly automated production systems are the backbone of the production of goods. They are used in a high variety of industrial sectors, e.g., automated packaging, production of chemicals, production of transportation vehicles. The systems are typically highly specialized and built on-demand.

Production systems contain mechanical, electrical and software parts and, hence, are jointly developed by engineers from the different disciplines. Examples of mechanical parts can vary from simple pipes to complex structures transforming energy into lateral movements.

Electrical components can be simple cables and sensors or complex power transmission systems taking electrical safety into account. Finally, the software mainly runs on programmable logic controllers and ensure the efficient and effective operation of the production system.

Automatic production systems is a relatively new term to describe those production systems which support new features like support for highly variable production up to lot size 1, are connected worldwide to other systems ranging from other production systems of the same company or suppliers to manufacturer's service databases. For example, the production systems could measure wear on its parts and use the communication capabilities to learn how to optimize the product quality related to the current wear.

Companies developing automatic production systems, like pure software companies, need to support continuous and fast delivery of customer value both in the short and long term. However, sustainable development can be hindered if both evolution and maintenance of the systems are hampered by Technical Debt. Technical Debt (TD) has been recently studied in the Software Engineering literature [1], [2], [3], [4]. The adaptation of the Technical Debt concept to automatic production systems has also been reported in a recent paper [5].

Technical Debt is composed of a debt, which is a sub-optimal technical solution that leads to short-term benefits as well as to the future payment of interest, which is the extra costs due to the presence of Technical Debt (for example, slow feature development or low quality) [6]. The principal is regarded as the cost of refactoring Technical Debt.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

XP '17 Workshops, May 22-26, 2017, Cologne, Germany
© 2017 Association for Computing Machinery.
ACM ISBN 978-1-4503-5264-2/17/05...\$15.00
<https://doi.org/10.1145/3120459.3120466>

A concrete example might be considered as the introduction of non-allowed dependencies in the systems, for example between component A that might access a private interface in another component B by skipping an existing standard API. The introduction of such dependency might save time: for example, implementing the call to the standard API might require additional work instead of calling a known interface. Skipping the standard (and optimal) solution will then save time in the short-term, which is considered as borrowing time with the consequence of having the sub-optimal solution in place (the debt). Fixing this dependency would also require some refactoring work in component A, which is considered as the principal of the debt. However, if the component B, with the standard API, would need to be replaced, when this happens, component A would need to be refactored. In the meanwhile, the code might have grown with new functionalities, which would cause the cost of repaying the debt in component A to be raised. Such long-term extra-cost is called interest.

Although accumulated Technical Debt might result in a gain between the short-term benefit and the interest paid, in many cases, documented in the literature and in software projects, the interest might largely surpass the gain, for example by leading to development crises [7]. There are several kinds of Technical Debt, such as Architecture Technical Debt, Testing Technical Debt, Source Code Technical Debt, etc. [3], depending on what artifact and on what level of abstraction the sub-optimality has occurred. In 2011 Guo et al. proposed an initial portfolio approach with the creation of Technical Debt items to be managed a further empirical study on tracking Technical Debt [8], in order to make the lack of quality and the risk visible for a software development team. Seaman et al. have identified the theoretical importance of Technical Debt as risk assessment tool in decision-making [9]. However, current literature does not tackle the introduction of Technical Debt management in large organizations developing automatic production systems, but only its conceptualization [5]. In this paper, we aim at providing a first quantification of the effort employed by the practitioners working with automatic production systems in managing Technical Debt.

The main question that we want to answer in this paper is:

RQ: How do companies developing automatic production systems deal with Technical Debt?

More specifically, we want to answer the following sub-questions:

RQ1: How much effort do companies developing automatic production systems spend on paying the interest (extra-costs)? On which kinds of Technical Debt?

RQ2: How much effort do companies spend in managing Technical Debt in automatic production systems?

RQ3: How much is Technical Debt known and tracked in a company developing automatic production systems?

RQ4: How familiar are practitioners in companies developing automatic production systems with the Technical Debt terminology?

In order to answer these questions, we conducted an exploratory survey involving 40 employees with different roles at an automatic production systems company, followed by a group interview with their managers.

The paper is organized as follows: in section II we introduce the methodology used in the survey, in section III we report the results of the survey, in Section IV we discuss the results and in section V we report our conclusions.

2. METHODOLOGY

In this research, a combination of quantitative and qualitative research approaches is used. The research is conducted by using methodological, source and observer triangulation. Methodological triangulation refers to the utilization of both qualitative and quantitative methods for gathering data and source triangulation refers to using different sources of data while observer triangulation refers using more than one observer in the study. Triangulation is important to increase the correctness of empirical research and therefore to provide a wider view. The following sections describe the methods used for conducting the survey and the supplementing qualitative follow-up interviews.

2.1 Research Design

We conducted an exploratory study at an international company developing automatic production systems. The company has sites in several parts of Europe. The company supplies complete systems for automation services with an extensive in-house software development of embedded, real-time software system.

We combined different sources of data (source triangulation) and different methodologies (methodology triangulation) for reliable results [10]. To fulfill these triangulation strategies, we conducted a survey among 40 participants. The different sources included 4 sub-organizations and different roles, i.e. developers, architects, and managers. To complement such quantitative and high-level investigation, we followed up with a qualitative, in-depth interviews with 8 managers at the automatic production systems company.

2.2 Survey

The web-survey was designed and hosted by the survey service called *SurveyMonkey*. The survey was using a combination of open- and closed-ended questions. To avoid bias in the survey, the questions were developed as neutral as possible, ordered in a way that one question did not influence the response to the next question.

A first draft of the web survey was tested by four industrial practitioners (developer, manager, project owner, and software architect) and by two Ph.D. students in order to evaluate the understanding of the questions. During this test, we also observed the time that was needed. The survey was made accessible between February and March 2016. The survey was anonymous, and participation in the survey was voluntary.

The first part of the survey gathered descriptive statistics to summarize the backgrounds of the respondents and their company. The second part of the survey included questions based on the previously presented research questions.

2.2.1 Survey Data Collection

In total, we obtained 40 complete answers from the companies. The survey was answered by 4 project managers, 3 product managers, 3 team leaders, 3 line managers, 1 R&D manager, 6 software architects and 20 software developers.

Technical Debt can affect different artifacts and phases of the development process. The different kinds of Technical Debt are reported in Table 1 together with their literature references.

Table 1: Kinds of Technical Debt Recognized in [3], [9], [10]

Survey entries / Challenges	Source and literature term
<i>Lack or low quality of testing</i>	<i>Test Debt [3]</i>
<i>Low code quality</i>	<i>Source Code Debt [3]</i>
<i>Lack or low quality of requirement</i>	<i>Requirement Debt [3]</i>
<i>Lack or low quality of documentation</i>	<i>Documentation Debt [3]</i>
<i>Dependency violations</i>	<i>Architecture Debt [3], [11]</i>
<i>Complex architectural design</i>	<i>Architecture Debt [3], [11]</i>
<i>Too many different patterns and policies</i>	<i>Architecture Debt [3], [11]</i>
<i>Dependencies to external resources/software</i>	<i>Architecture Debt [3], [11]</i>
<i>Lack of reusability in design</i>	<i>Architecture Debt [3], [11]</i>
<i>Uneasy/Tensed social interactions between different stakeholders</i>	<i>Social Debt [3], [12]</i>
<i>Lack of adequate environment and infrastructure during development</i>	<i>Infrastructure Debt [3]</i>

In the second part of the survey, we asked the main six questions:

Q1: “Which of the following challenges generate the most negative impact on your daily software development work? Please rank them from 1 to 11.”

Q2: “How much of the overall development time is wasted because of these issues?”

Q3: “How much of the overall development effort is usually spent on Technical Debt management activities?”

Q4: “I track (using tools, documentation, etc.) Technical Debt in our system.”

Q5: “If you track Technical Debt in your project, what kind of tool(s) do you use?”

Q6: “How familiar are you with the term “Technical Debt”?”

For questions Q2 and Q3, since we wanted to quantify the amount of effort related to the interest and the management of Technical Debt, we provided a scale including the following options: “<10%”, “10-20%”... “80-90%” and “I don’t know.”

For Q4 we asked the respondents to report their agreement on a 6-steps Likert scale: “strongly disagree”, “disagree”, “somewhat disagree”, and the symmetric scale for agreement.

As for question Q5, we asked the participants to report the tools used in a qualitative way (text-box).

Finally, for Q6 we provided the answers “Not at all familiar”, “Slightly familiar”, “Moderately familiar”, “Very familiar” and “Extremely familiar”.

2.2.2 Survey Data Analysis

We have analyzed the answers from Q2 and Q3 in order to understand the magnitude of the estimated effort spent by the respondents on the interest and management of Technical Debt (RQ1 and RQ2). We transformed the answers from categorical to numerical: “<10%” to 5, “10-20%” to 15 and so on. When calculating the means, we did not consider the “I don’t know” answers.

We applied frequency analysis on Q4: the intention was to understand how much the studied company was tracking Technical Debt (RQ3). In order to do so, we transformed the categorical data to a 6 Likert scale 1-6, where “strongly disagree” was mapped to 1 and “strongly agree” to 6. We also reported the grouped answers in 3 main intervals, “No tracking” {1-2}, “Somewhat tracking” {3-4}, and “Tracking” {5-6}. We also qualitatively analyzed the answers in the text field for Q5 to understand what tools were used to track Technical Debt.

Finally, we analyzed the frequencies of the respondents’ answers to Q6 in order to understand how much they were familiar with Technical Debt (RQ4).

3. RESULTS

This section presents the results for the research question presented in Section I, and the results are grouped according to each research questions.

3.1 Survey Results

RQ1: *How much effort do companies developing automatic production systems spend on paying the interest (extra-costs)? On which kinds of Technical Debt?*

Technical Debt has extra costs that are associated with its interest, or else those extra activities that would not have been done if the Technical Debt was not present.

The first analysis of the survey data shows that the company waste **32%** of the development process, on average, because of the interest (extra-costs) generated by Technical Debt. To calculate the 32%, we parsed each answer by picking the middle point between the suggested effort span: for example, if an answer was “20-30%” we parsed it to 25.

Then we calculated the average. This would give a good overview, on average, on how much costs has Technical Debt generated. However, we report the distribution of the answers as shown in Figure 1. We can see how most of the answers are clustered between 10 % and 50 %, especially between 20 % and 40 %. Only three respondents could not answer this question. In picture Figure 2, we show the results of the analysis of the ranking reported by the respondents. These results help understanding which challenges generated the extra costs reported earlier.

The value, for each Technical Debt kind, is the average of the weights associated with the ranking reported by the respondents. For example, if one of the Technical Debt kinds were ranked as most hurtful by one respondent, it would be given a score of 11 (the first out of 11 kinds), if second 10 and so on, with the weight 1 if ranked as the last one. Given these weights, a mean value was calculated for each Technical Debt kind: this way, we could see which challenges were ranked as the most hurtful ones.

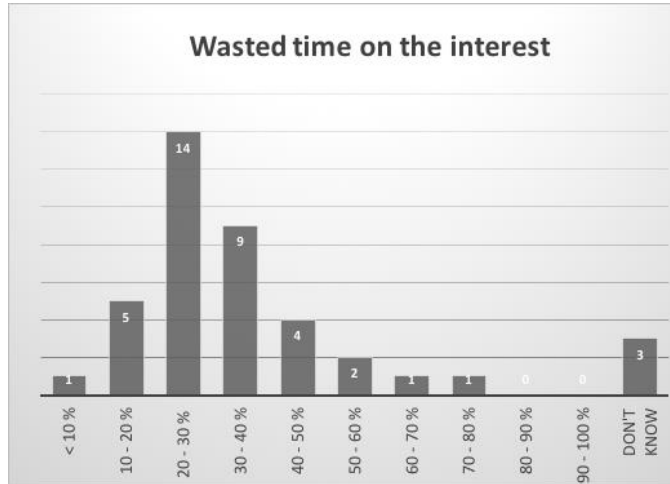


Figure 1. The graph shows how much time is, on average, spent by the practitioners on paying the Interest (extra-cost) associated with Technical Debt

In Figure 2, it seems that Technical Debt would have a major impact on the extra costs (interest) due to Requirement TD and Testing TD, while Documentation, Architecture (complex and lack of reusability), and Code have been reported as quite important as well. There is quite a difference between these kinds and the last five kinds, which have been reported as less hurtful. Also, it is possible to see that the percentiles boxes are spanning quite a lot of positions across the whole spectrum. Especially for the 25-percentile, the answers cover the entire spectrum, which means that the respondents ranked these kinds in quite different ways. However, the 50-percentiles and the medians represent a good indication of which ones were ranked the most hurtful.

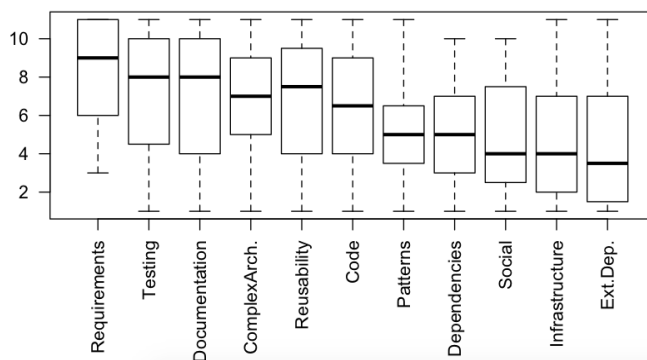


Figure 2. The picture shows which Technical Debt kinds have been ranked as the most hurting. The black lines are the median, the white solid boxes contain 50% of the respondents, while the dashed lines show how 25% of the answers span different options.

RQ2: How much effort do companies spend in managing Technical Debt in automatic production systems?

Besides the cost of the debt's interest, there are other activities (and therefore costs) related to Technical Debt. These are *management activities* necessary to keep Technical Debt at bay: for example, repaying the debt (refactoring the code), finding the debt, measuring it, understanding the issues, and finally managing, keeping track and communicating the debt that cannot be refactored. All these activities account for an additional cost.

This additional managerial extra-cost has been identified as being, on average, **24 %** of the development time. This number has been calculated using the same procedure used to calculate the interest (see the previous section).

The answers related to this extra-cost are reported in Figure 3. We can see here how most of the answers lie between 10 % and 40 % of development time.

RQ3: How much is Technical Debt known and tracked in a company developing automatic production systems?

Having understood the costs related to Technical Debt in the company, we wanted to understand how much Technical Debt is known and actively tracked by the practitioners.

The graph in Figure 4 shows how many of the practitioners are aware of the Technical Debt in their system as well as how much they actively track it. It is important to understand if they use systematic practices to keep track of Technical Debt or if it is managed in an ad-hoc way.

The comparison shows that the practitioners are in general moderately aware of how much Technical Debt they have in their systems (the average is 3.68, which is between "Somewhat Agree" and "Somewhat Disagree"), although this awareness varies greatly. As for the systematic tracking, this does not happen a lot, as most of the respondents answered "disagree" (on different levels). This shows that Technical Debt is moderately known even without tracking it.

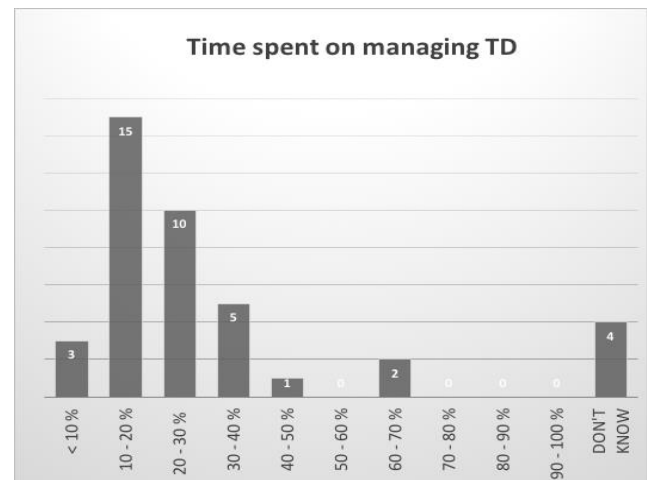


Figure 3. The graph shows how much time is, on average, spent by the practitioners on managerial activities related to Technical Debt

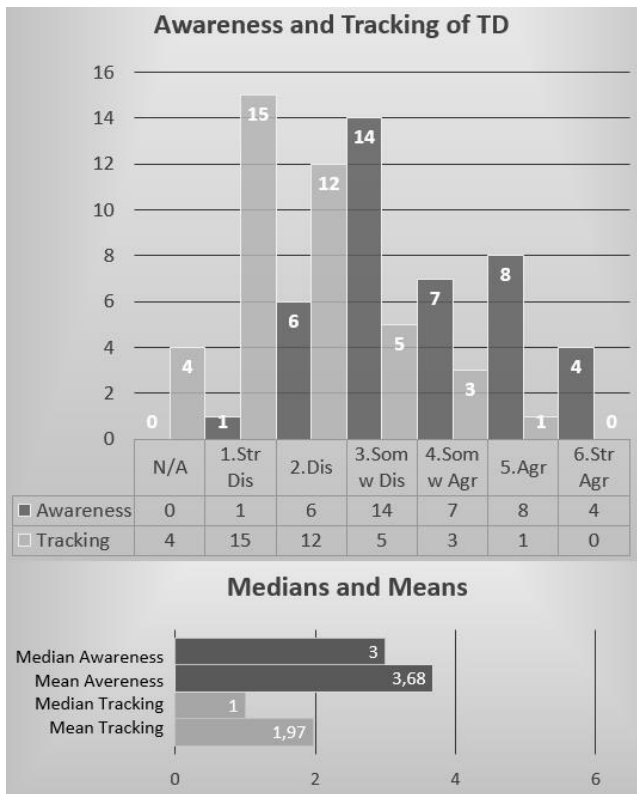


Figure 4. The picture shows the compared distributions of the answers both for awareness and tracking of Technical Debt. The means are also compared.

As for the few tools used for tracking Technical Debt, the practitioners report Jira and Excel.

Both these tools are not optimized to report Technical Debt, as one is a generic issue tracker without support for Technical Debt related fields (such as interest, principal, etc.). As for Excel, it is a generic purpose tool that can be customized if the users have good knowledge of Technical Debt already (which was not the case in this company for many respondents). This means that the practitioners do not track Technical Debt in an efficient and effective way also because of the lack of proper tools to support such activity.

RQ4: How familiar are practitioners in companies developing automatic production systems with the Technical Debt terminology?

By looking at the familiarity of the practitioners with the terminology related to Technical Debt, it seems that it varies a lot in the group. There are more respondents on the less familiar side of the graph.

This is confirmed by the average and the median, both of which lie close to “slightly familiar”. However, as many as ten respondents (25% of the sample) are instead very familiar with the concept. This variance would require further analysis and investigation to understand what it depends on.

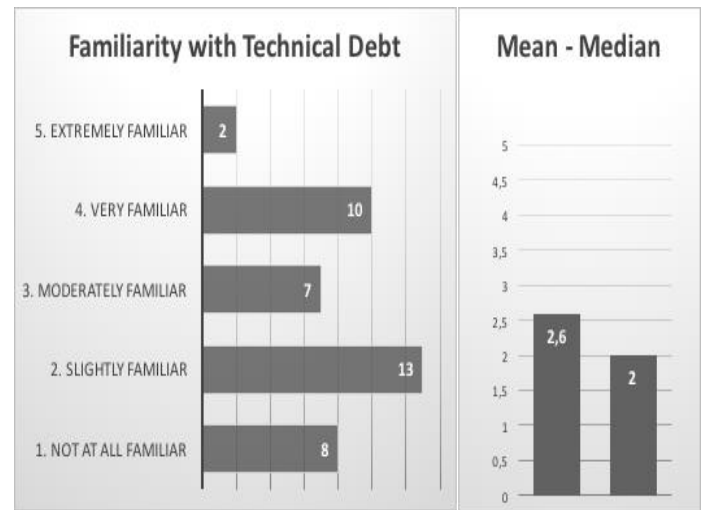


Figure 5. The graph shows the distribution of the respondents with respect to different levels of familiarity with the Technical Debt concept. We also report the mean of these answers.

3.2 Interview Results

When we discussed the results with the managers at the company, we received mixed feedback. Some of the managers thought that the results would represent the current situation, while some other were somewhat surprised to see such a high price paid for sub-optimal quality. Especially, they were surprised when they saw a few of the respondents having to deal with Technical Debt for 70 % of their time.

The managers commented that the time spent in paying the interest on Technical Debt and managing it would be better spent in doing more features and raising the reliability of the software.

As for the management part, we understood that there is no time associated with explicit Technical Debt management. This activity is carried out individually by practitioners according to their knowledge and awareness. It is, therefore, an ad-hoc practice, and it is not consistent in different parts of the company.

4. DISCUSSION

This section presents and discusses the findings, and the implications of the research questions presented in Section I. The findings are grouped and discussed according to each research questions.

RQ1: How much effort do automatic production systems companies spend on paying the interest on Technical Debt? On which kinds of Technical Debt?

The company wastes quite a lot of effort in paying the interest on Technical Debt. The average interest is around 32%, which is a substantial amount of resources.

From the data, it seems that, in this company developing automatic production systems, the most common kind of Technical Debt is Requirement Debt, followed by Testing Debt. However, Documentation Debt, Architecture Debt (especially Complexity and lack of Reusability) and Source Code Debt are also quite important.

RQ2: How much effort do automatic production systems companies spend in managing Technical Debt?

Despite the high interest paid, the company also pays quite a lot of resources in trying to repay Technical Debt and to keep the interest low. The average management effort is estimated to be about 24%. Combined with the previous datum, we can see how the automatic production systems company spends a lot of resources in paying the interest and managing Technical Debt.

RQ3: How much is Technical Debt known and tracked by an automatic production systems company?

Employees in the automatic production systems company are, on average, moderately aware of the Technical Debt in their system: the average is between “somewhat disagree” and “somewhat agree” (mean of 3.68). However, very few of them track Technical Debt, as the average tracking answer is “Disagree” (1.9). Would the improvement in tracking Technical Debt increase the awareness of Technical Debt? Would tracking Technical Debt in a systematic way decrease the resources spent on Technical Debt management and, more importantly, would it reduce the interest paid by the company? Future research should assess if the introduction of Technical Debt management in a company developing automatic production systems improved the current long-term negative effects of Technical Debt.

RQ4: How familiar are practitioners in companies developing automatic production systems with the Technical Debt terminology?

The knowledge of Technical Debt in the company varies a lot. Perhaps one of the hinders in systematically tracking Technical Debt is that it is not visible and recognized as a first-class entity such as bugs and features, as reported in [13]. Since the Technical Debt metaphor is a means to communicate the risk of poor quality among the different stakeholders in a company, a shared knowledge of such concept and its terminology would allow a better recognition as well as a better management.

Implication for industrial practices

The results here show that the current state of practice is quite mixed even within the same companies. Some practitioners know more about Technical Debt, some others are not familiar at all. We see this as an opportunity for companies developing automatic production systems to learn about Technical Debt and to make their employees aware of the concept, the terminology, and its use. This could increase the awareness of the Technical Debt as a first-class concept in the (software) development, and would also increase the opportunities for a systematic management as well as a suitable communication of the risks related to it.

Although practitioners have, on average, a moderate awareness of Technical Debt even if without systematically tracking, this awareness can be increased. If we take into consideration the financial meaning of the Technical Debt, an organization would want to know what the debt is taken on and how much interest is paid on the debt that has been taken. This is important to strategically act on this information, for example, to refactor the debt that has a high interest. In this perspective, a moderate knowledge of the existing Technical Debt is quite suboptimal, as it does not allow to know debt and interest, which can bear a high risk of productivity loss or other extra costs.

Despite the reported knowledge of Technical Debt from some of the respondents, it seems that tracking is not well spread in the

company. The main reasons are that Technical Debt is not recognized at a company level, which does not allow for strategic actions, and that the available tools do not support Technical Debt management. This is also reported by other studies, such as [13] and [14].

In summary, we see a need for practical improvement of Technical Debt management consisting in raising the awareness of Technical Debt, increasing its tracking, which consequently entails a better visualization and strategic management, as well as the introduction of Technical Debt -specific tools to make such task supported in the organizations.

Theoretical implications

The results in this paper support the hypothesis, reported in [5], that Technical Debt is a concept applicable to companies developing automatic production systems, and not only to software companies. This implies that more investigation should be conducted at companies developing automatic production systems to understand if the Technical Debt spans more disciplines (for example if the Technical Debt can be applied to mechanical engineering) and especially if Technical Debt in one discipline (e.g. software) would generate extra costs in other disciplines (e.g. electrical engineering) and vice-versa. In this paper, we report the existence of the overall costs and challenges, but a more in-depth qualitative investigation would reveal important details on how the different disciplines interact with each other in a Technical Debt perspective.

In comparison with the survey reported in [14], we can see how the state of the practice is quite similar between generic software companies and companies developing automatic production systems. Although the practitioners report similar prior knowledge and familiarity with the concept of Technical Debt, the actual source of Technical Debt varies between the two surveys: in the company developing automatic production systems, it seems that the worse extra costs paid because of Technical Debt are a requirement and test level, while in the more generic software development the first choice as the most hurtful is architecture. This difference could be due to the interaction between different disciplines. The combination of software artifacts together with hardware requires clear interfaces between the disciplines: software modules need to conform (and to be tested) according to requirements that are bounded to hardware specifications. Such requirements might not be so stringent in a “pure” software system, in which this sub-optimality would not create as much extra cost as it does in automatic production systems.

Threats to validity and limitations

A major threat usually present in the questionnaire is related to construct validity: in this survey, we needed to be sure that “Technical Debt” was well understood by the practitioners, as they might not be familiar with it (as the results show). To mitigate this threat, we have not mentioned “Technical Debt” directly (but in the last two questions), but we have used operationalized definitions of different kinds of debt according to the different kinds: for example, we have represented Architectural Debt as “Complex Architecture”, “Too many patterns and policies”, etc. Another threat to validity is related to the costs calculated in this study. The reported percentages might not be precise, as they have been calculated by picking the middle points. Therefore, the reported averaged results might deviate by

± 5 % points, although this span is quite unlikely. Finally, about external validity threats, the study was exploratory and has involved only one company. This might not be representative of the majority of companies developing automatic production systems. For better generalizable results, more companies should be studied. Finally, to mitigate reliability biases, we have employed source triangulation, methodology triangulation and, at last, observer triangulation.

5. CONCLUSION

The company developing Automatic Production Systems that we have investigated spends quite a lot of resources in paying the interest on Technical Debt, on average 32 % of the development time, although a similar amount of resources are employed in trying to manage Technical Debt and to reduce the interest. One of the reasons might be that Technical Debt, even if its awareness is moderate, is not systematically managed in the company (very few employees track Technical Debt). The current tools and practices don't support a better strategic management of Technical Debt. These results reveal that Technical Debt in automatic production systems companies is a big challenge. Systematically managing Technical Debt would probably improve the situation, and an attempt in such direction should be studied in order to understand what increases the overall management of Technical Debt. Another target is to understand if Technical Debt is related to software only, or if it spans other disciplines as well. Our results suggest that there is a difference in what kinds of Technical Debt generate more effort, and this might be related to the combination of software and hardware development. Finally, more automatic production systems companies would need to be studied in order to understand how generalizable the results are.

6. ACKNOWLEDGMENTS

We thank the industrial partners of the Software Center for the collaboration and the contribution of this study.

7. REFERENCES

- [1] Besker, T, Martini, A, Bosch, J., A systematic literature review and a unified model of Architectural Technical Debt, in: Euromicro Conference Series on Software Engineering and Advanced Applications (SEAA), 2016.
- [2] Brown, N., Cai, Y., Guo, Y., Kazman, R., Kim, M., Kruchten, P., Lim, E., MacCormack, A., Nord, R., Ozkaya, I., others, 2010. Managing technical debt in software-reliant systems, in: Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research. ACM, pp. 47–52.
- [3] Cunningham, W., 1992. The WyCash portfolio management system, in: ACM SIGPLAN OOPS Messenger. ACM, pp. 29–30.
- [4] Ernst, N.A., Bellomo, S., Ozkaya, I., Nord, R.L., Gorton, I., 2015. Measure It? Manage It? Ignore It? Software Practitioners and Technical Debt, in: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2015. ACM, New York, NY, USA, pp. 50–60. doi:10.1145/2786805.2786848
- [5] Guo, Y., Seaman, C., Gomes, R., Cavalcanti, A., Tonin, G., Da Silva, F.Q., Santos, A.L.M., Siebra, C., 2011. Tracking technical debt—An exploratory case study, in: Software Maintenance (ICSM), 2011 27th IEEE International Conference on. pp. 528–531.
- [6] Kruchten, P., Nord, R.L., Ozkaya, I., 2012. Technical Debt: From Metaphor to Theory and Practice. IEEE Softw. 29, 18–21. doi:10.1109/MS.2012.167
- [7] Li, Z., Avgeriou, P., Liang, P., 2015. A systematic mapping study on technical debt and its management. J. Syst. Softw. 101, 193–220. doi:10.1016/j.jss.2014.12.027
- [8] A. Martini and J. Bosch, "The Danger of Architectural Technical Debt: Contagious Debt and Vicious Circles," in Proceedings - 12th Working IEEE/IFIP Conference on Software Architecture, WICSA 2015, 2015, pp. 1-10.
- [9] Martini, A., Bosch, J., Chaudron, M., n.d. Investigating Architectural Technical Debt Accumulation and Refactoring over Time: a Multiple-Case Study. Inf. Softw. Technol. doi:10.1016/j.infsof.2015.07.005
- [10] Runeson, P., Höst, M., 2008. Guidelines for conducting and reporting case study research in software engineering. Empir. Softw. Eng. 14, 131–164. doi:10.1007/s10664-008-9102-8
- [11] Seaman, C., Guo, Y., Zazworka, N., Shull, F., Izurieta, C., Cai, Y., Vetro, A., 2012. Using technical debt data in decision making: Potential decision approaches, in: 2012 Third International Workshop on Managing Technical Debt (MTD). Presented at the 2012 Third International Workshop on Managing Technical Debt (MTD), pp. 45–48. doi:10.1109/MTD.2012.6225999
- [12] Tamburri, D.A., Kruchten, P., Lago, P., Vliet, H. van, 2013. What is social debt in software engineering?, in: 2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE). Presented at the 2013 6th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE), pp. 93–96. doi:10.1109/CHASE.2013.6614739
- [13] Tom, E., Aurum, A., Vidgen, R., 2013. An exploration of technical debt. J. Syst. Softw. 86, 1498–1516. doi:10.1016/j.jss.2012.12.052
- [14] Vogel-Heuser, B., Rösch, S., Martini, A., Tichy, M., 2015. Technical Debt in Automated Production Systems. Proc. Int. Conf. Softw. Maint. Ecolution Brem. Ger.